Unit 1. Simple Programs

1.0 First Program

Sue is home for the Christmas holiday when her mother asks her to fix a "computer problem." It turns out that the problem is not the computer itself, but some data their bank has sent them. Instead of e-mailing a list of stock prices in US dollars ($), the entire list is in Euros (€)! Rather than perform the conversion by hand, Sue decides to write a program to do the conversion. Without referencing any books (they are back in her apartment) or any of her previous programs (also back in her apartment), she quickly writes the code to complete the task.

Objectives

By the end of this class, you will be able to:

- Use the provided tools (Linux, `emacs`, `g++`, `styleChecker`, `testBed`, `submit`) to complete a homework assignment.
- Be familiar with the University coding standards (Appendix A. Elements of Style).

Prerequisites

Before reading this section, please make sure you are able to:

- Type the code for a simple program (Chapter 0.2).

Overview of the process

The process of turning in a homework assignment consists of several steps. While these steps may seem unfamiliar at first, they will be well-rehearsed and second-nature in a week or two. The lab assistants (wearing green vests in the Linux lab) are ready and eager to help you if you get stuck on the way. The process consists of the following steps:

1. Log into the lab
2. Copy the assignment template using `cp`
3. Edit your file using `emacs`
4. Compile the program using `g++`
5. Verify your solution with `testBed`
6. Verify your style with `styleChecker`
7. Turn it in with `submit`

This entire process will be demonstrated in "Example – Hello World" at the end of the chapter.

0. Login

All programming assignments are done on the Linux system. This includes the pre-class assignments, the projects, and the in-lab tests. You can either go to the Linux Lab to use the campus computers, or connect remotely to the lab from your personal computer. Either way, you will need to log in. If you have not done this in Assignment 0.0, please re-visit the quiz for the default password. The lab assistants will be able to help you reset your password if necessary. Please see Appendix C: Lab Help for a description of what the lab assistants can and cannot do.

It is worthwhile to set up your computer so you do not need to come to the lab to do an assignment. The method is different for a Microsoft Windows computer than it is for an Apple Macintosh computer.

Remote access for Windows computers

1. Download the tool called PuTTY

   Setup - PuTTY

2. Go to the lab and read the IP address (four numbers separated by periods) from any machine in the lab. They are 157.201.194.201 through 157.201.194.210. This will be the physical machine you are accessing when using remote access
3. Boot PuTTY and type in your IP address from step 2 and the port 215. You might want to save this session so you don't have to keep typing the numbers in.
4. Select [OPEN]. After you specify your username and password, you are now logged into that machine.

Remote access for Macintosh or Linux computers

If you are on a Macintosh or a Linux computer, bring up a terminal window and type the following command:

```
ssh  <username>@<ip>  -p  215
```

If, for example, you want to connect to machine `157.201.194.230` and your username is "`sam`", then you would type:

```
ssh  sam@157.201.194.230  -p  215
```

For more information, please see:

Setup - Terminal

1. Copy Template

Once you have successfully logged into the Linux system (either remotely or in the Linux Lab), the next step is to copy over the template for the assignment. All the assignments for this class start with a template file which has placeholders for the assignment name and the author (that would be you!). This file, and all other files pertaining to the course, can be found on:

```
/home/cs124
```

The assignment (and project and test) template is located on:

```
/home/cs124/template.cpp
```

On the Linux system, we type commands rather than use the mouse. The command used to copy a file is called "`cp`". The syntax for the copy command is:

```
cp  <source file>  <destination file>
```

If, for example, you were to copy the template from `/home/cs124/template.cpp` into `hw10.cpp`, you would type the following command:

```
cp  /home/cs124/template.cpp  hw10.cpp
```

Most Linux commands do not display anything on the screen if they were successful. You will need to do a directory listing (`ls`) to see if the file copied. A list of other common Linux commands are the following:

Navigation tools	`cd` ..... Change Directory `ls` ..... List information about file(s) `cat` ..... Display the contents of a file to the screen `clear` ..... Clear terminal screen `exit` ..... Exit the shell `yppasswd` ..... Modify a user password
Organization tools	`mkdir` ..... Create new folder(s) `mv` ..... Move or rename files or directories `rm` ..... Remove files
Programming tools	`emacs` ..... Common code editor `vi` ..... More primitive but ubiquitous editor `g++` ..... Compile a C++ program
Homework tools	`styleChecker` . Run the style checker on a file `testBed` ..... Run the test bed on a file `submit` ..... Turn in a file

For more commands or more details on the above, please see Appendix D: Linux and Emacs Cheat-Sheet.

Sue's Tips

Be careful how you name your files. By the end of the semester, you could easily get lost in a sea of files. Spend a few moments thinking of how you will organize all your files as this will be a useful practice for the remainder of your career.

2. Edit with Emacs

Once the template has been copied to your directory, you are now ready to edit your program. There are many editors to choose from. Some editors are specialized to a specific task (such as Excel and Photoshop). The editor we use for programming problems is specialized for writing code. There are many editors you may use, including emacs and vi. For help with common `emacs` commands, please see "Appendix D: Linux and Emacs Cheat Sheet."

If you would like to write a program in `hello.cpp`, you can use emacs to edit create and edit the file with:

```
emacs hello.cpp
```

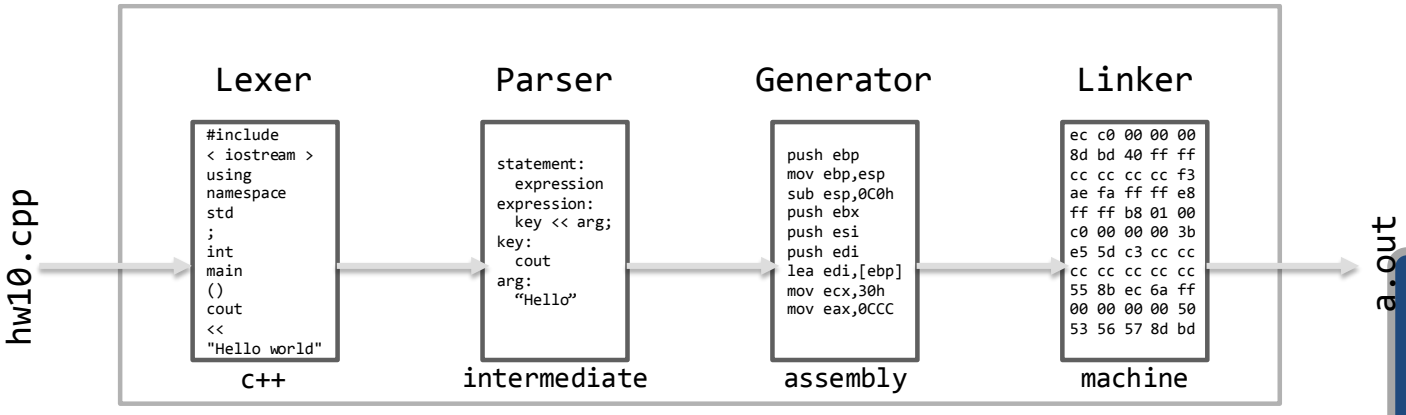
This will start emacs with a blank document named `hello.cpp`. From here you can type anything you like. However, if you wish this program to function correctly, you need to type valid C++. For your first program, you can make it say "Hello World" as we need to do for the first assignment:

```
/**************************************************************************
* Program:
*    Assignment 10, Hello World
*    Brother Helfrich, CS124
* Author:
*    Sam Student
* Summary:
*    This program is designed to be the first C++ program you have ever
*    written. While not particularly complex, it is often the most difficult
*    to write because the tools are so unfamiliar.
**************************************************************************/

#include <iostream>
using namespace std;

/**************************************************************************
* Hello world on the screen
**************************************************************************/
int main()
{
   // display
   cout << "Hello World\n";

   return 0;
}
```

When you have finished writing the code for your program, save it and exit the editor. To save, first hit the `<control>` and `x` key at the same time, followed shortly with `<control>` and `s`. The shorthand for this key sequence is `C-x C-s`. You can then exit emacs with `C-x C-c`. More emacs keystokes are presented in Appendix D at the back of this book.

3. Compile

After the program is saved in a file, the next step is compilation. Compilation is the process of translating the program from one format (C++ in this case) to another (machine language). This process is remarkably similar to how people translate text from French to English. There are four steps:



1. **Lexer**: Lexing is the process of breaking a list of text or sounds into words. When a non-speaker hears someone speak French, they are not even sure how many words are spoken. This is because they do not have the ability to lex. The end result of the lexing process is a list of tokens or words, each hopefully part of the source language.
2. **Parser**: Parsing is the process of fitting the words or tokens into the syntax of the language. In French, that is the process of recognizing which word is the subject, which is the verb, and which is the direct object. Once the process of parsing is completed, the listener understands not only what the words are, but what they mean in the context of the sentence.
3. **Generator**: After the meaning of the source language is understood through the parsing process, the next step is to generate text in the target language. In the case of the French to English translation, this means putting the parsed meaning from the French language into the equivalent English words using the English syntax. In the case of compiling C++ programs, the end result of this phase is assembly language similar to what we used in Chapter 0.2.
4. **Linker**: The final phase is to output the result from the code generator into a format understood by the listener. In the case of the French to English translation, that would involve speaking the translated text. In the case of compiling C++ code, that involves creating machine language which the CPU will be able to understand.

All four of these steps are done almost instantly with the compiler. The compiler we use in this class is `g++`. The syntax is:

```
g++  <source file>
```

If, for example, we are going to compile the file `hw10.cpp`, the following command will need to be typed:

```
g++  hw10.cpp
```

If the compilation is successful, then the file `a.out` will be created. If there was an error with the program due to a typographical error, then the compiler will state what the error was and where in the program the error was encountered.

4. Test Bed

After we have successfully passed the compilation process, it is then necessary to verify our solution. This is typically done in a two-step process. The first is to simply run the program by hand and visually inspect the output. To execute a newly-compiled program, type the name of the program in the terminal. Since the default name of a newly-compiled program is "`a.out`," then type:

```
a.out
```

The second step in the verification process is to test the program against the key. This is done with a program called Test Bed. Test Bed compares the output of your program against what was expected. If everything behaves correctly, a message "No Errors" will be displayed. On the other hand, if the program malfunctions or produces different output than expected, then the difference is displayed to the user. In this way, Test Bed is a two-edged sword: you know when you got the right answer, but it is exceedingly picky. In other words, Test Bed will notice if a space was used instead of a tab even though it appears identical on the screen. The syntax for Test Bed is:

```
testBed  <test name>  <file name>
```

The first parameter to the Test Bed program is the test which is to be run. This test name is always present on a homework assignment, in-lab test, and project. The second parameter is the file you are testing. If, for example, your program is in the file `hw10.cpp` and the test is `cs124/assign10`, then the following code will be executed:

```
testBed  cs124/assign10  hw10.cpp
```

It is important to note that you will not get a point on a pre-class assignment unless Test Bed passes without error.

5. Style Checker

Once the program has been written and passes Test Bed, it is not yet finished. Another important component is whether the code itself is human-readable and in a standard format. This is collectively called "style." A programming style consists of many components, including variable names, indentations, and comments.

While style is an inherently subjective notion, we have a tool to help us with the process. This tool is called Style Checker. While Style Checker will certainly not catch all possible style mistakes, it will catch the most obvious ones. You should never turn in an assignment without running Style Checker first. The syntax for Style Checker is:

```
styleChecker  <file name>
```

If, for example, you would like to run Style Checker on `hw10.cpp`, then the following command is to be executed.

```
styleChecker  hw10.cpp
```

The main components to style include:

Variable names	Variable names should completely describe what each variable contains. Each should be camelCased: capitalize the first letter of every word in the name except the first word. We will learn about variables in Chapter 1.2: `numStudents`
Function names	Function names are camelCased just like variable names. Function names are typically verbs while variable names are nouns. We will learn about functions in Chapter 1.4. `displayBudget()`

Indent	Indentations are three spaces. No tabs please!
	`{`
	`   cout << "Hello world\n";`
	`}`

Line length	Lines are no longer than 80 characters in length. If more space is needed for a comment, break the comment into two lines. The same is true for `cout` statements (Chapter 1.1) and function parameters (Chapter 1.4).
	`// Long comments can be broken into two lines`
	`// to increase readability. Start each new`
	`// line with "//"s`

Program comments	All programs have a program comment block at the beginning of the file. This can be found in the standard template. An example is:
	`/*****************************************`
	`* Program:`
	`*    Assignment 10, Hello World`
	`*    Brother Helfrich, CS124`
	`* Author:`
	`*    Sam Student`
	`* Summary:`
	`*    Display a message`
	`*****************************************/`

Function comments	Every function such as `main()` has a comment block describing what the function does:
	`/*****************************************`
	`* MAIN`
	`* This program will display a simple message`
	`* on the screen`
	`*****************************************/`

Space between operators	All operators, such as addition (`+`) and the insertion operator (`<<`) are to have a single space on either side to set them apart: `sumOfSquares += userInput * userInput;`

For more details on the University's style guidelines, please see "Appendix A: Elements of Style" and look at the coding examples presented in this class.

6. Submit

The last step of turning in an assignment is to submit it. While we discuss this as the end of the homework process, you can submit an assignment as often as you like. In the case of multiple submissions, the last one submitted at the moment the assignment is graded is the one that will be used. It is therefore a good idea to submit your assignments frequently so your professor has the most recent copy of your work. The syntax for the program submission tool is:

```
submit  <file name>
```

If, for example, your program is named "`hw10.cpp`," then the following command is to be executed:

```
submit  hw10.cpp
```

One word of caution with the Submit tool. The tool reads the program header to determine the professor name, the class number, and the assignment number. If any of these are incorrect, then the program will not be submitted correctly. For example, consider the following header:

```
/*************************************************************************
* Program:
*    Assignment 10, Hello World
*    Brother Helfrich, CS124
* Author:
*    Susan Bakersfield
* Summary:
*    This program is designed to be the first C++ program you have ever
*    written. While not particularly complex, it is often the most difficult
*    to write because the tools are so unfamiliar.
*************************************************************************/
```

Here, Submit will determine that the program is an `Assignment` (as opposed to a `Test` or `Project`), the assignment number is `10`, the professor is Br. `Helfrich`, and the class is `CS 124`. If any of these are incorrect, then the file will be sent to another location. To help you with this, submit tells the user what it read from the header:

```
submit homework to helfrich cs124 and assign10. (y/n)
```

It is worthwhile to read that message.

Sam's Corner

Submit is basically a fancy copy function. It makes two copies of the program: one for you and one for the instructor. If, for example, you submitted to "Assignment 10" for "CS 124", then you will get a copy on.

```
/home/<username>/submittedHomework/cs124_assign10.cpp
```

Observe how the name of the file is changed to that of the assignment and class name. The second copy gets sent to the instructor. Here the filename is changed to the login ID. If, for example, your login is "`eniac`", then the file appears as eniac.cpp in the instructor's folder.

Please do not use a dot in the name of your file. If you submit `hw1.0.cpp`, for example, then it will appear as `eniac.0` instead of `eniac.cpp` and the instructor will not grade it

Example 1.0 – Display "Hello World"

Demo

This example will demonstrate how to turn in a homework assignment. All the tools involved in this process, including `emacs`, `g++`, `testBed`, `styleChecker`, and `submit`, will be illustrated.

Problem

Write a program to prompt to display a simple message on the screen. This message will be the classic "Hello World" that we seem to always use when writing our first program with a new computer language.

Solution

The code for the solution is:

```
/**********************************************************************
* Program:
*    Assignment 10, Hello World
*    Brother Helfrich, CS124
* Author:
*    Sam Student
* Summary:
*    This program is designed to be the first C++ program you have ever
*    written. While not particularly complex, it is often the most difficult
*    to write because the tools are so unfamiliar.
**********************************************************************/

#include <iostream>
using namespace std;

/**********************************************************************
* Hello world on the screen
**********************************************************************/
int main()
{
   // display
   cout << "Hello World\n";

   return 0;
}
```

Of course the real challenge is using the tools…

Challenge

As a challenge, modify this program to display a paragraph including your name and a short introduction. My paragraph is:

```
Hello, I am Br. Helfrich.

My favorite thing about teaching is interacting with interesting students every
day. Some days, however, students have no questions and don't bother to come by
my office. Those are long and lonely days...
```

See Also

The complete solution is available at 1-0-firstProgram.cpp or:

```
/home/cs124/examples/1-0-firstProgram.cpp
```

Problem 1

If your body was a computer, select all the von Neumann functions that the spinal cord would perform?

Answer:

_____

Please see page 5 for a hint.

Problem 2

If a given processor were to be simplified to only contain a single instruction, which part would be most affected?

Answer:

_____

Please see page 5 for a hint.

Problem 3

Which of the following does a CPU consume? {Natural language, C++, Assembly language, Machine }?

Answer:

_____

Please see page 5 for a hint.

Problem 4

What is wrong with the following program:

```
#include <iostream>
using namespace std;

int main()
(
   cout << "Howdy\n";

   return 0;
)
```

Answer:

_____

Please see page 7 for a hint.

Assignment 1.0

Write a program to put the text "`Hello World`" on the screen. Please note that examples of the code for this program are present in the course notes.

Example

Run the program from the command prompt by typing `a.out`.

```
$a.out
Hello World
$
```

Instructions

Please…

1. Copy template from: `/home/cs124/template.cpp`. You will want to use a command like:

   ```
   cp /home/cs124/template.cpp assignment10.cpp
   ```

2. Edit the file using `emacs` or another editor of your choice. For example:

   ```
   emacs assignment10.cpp
   ```

3. After you have typed your program, save it and compile with:

   ```
   g++ assignment10.cpp
   ```

4. If there are no errors, you can run it with:

   ```
   a.out
   ```

   Please verify your solution against test-bed with:

   ```
   testBed cs124/assign10 assignment10.cpp
   ```

5. Check the style to ensure it complies with the University's style guidelines:

   ```
   styleChecker assignment10.cpp
   ```

6. Turn your assignment in with the `submit` command. Don't forget to submit your assignment with the name "`Assignment 10`" in the header

   ```
   submit assignment10.cpp
   ```

Please see page 21 for a hint.